ALLIANCE UNIVERSITY

Private University established in Karnataka State by Act No.34 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi

Alliance School of Advanced Computing

Course Name: GRAPH ANALYTICS

Course Code: E1ITC 302

Assignment No: 5

Name of the Student: Gali Lakshmi prasanna

Registration Number:2411021060312

A. Title of The Problem:

University Student Performance Analysis Using Neo4j

B. Problem Statement in details:

Problem Statement: University Course Registration System Using Neo4j

A university must maintain information about students, faculty members, courses, and academic departments. As the number of students and courses grows, handling the connections between these entities can become difficult.

This project uses Neo4j, a graph database, to organize university information by representing students, faculty members, courses, and departments as nodes connected through relationships. Students can enroll in courses, faculty members can teach different courses, and each course can be associated with a particular department.

With the help of Cypher queries, the system can easily find details such as the courses selected by each student, the faculty members responsible for teaching specific courses, and the courses available under each department.

The primary goal of this project is to show how graph databases make it easier to manage connected data, retrieve related information quickly, and improve the organization of university academic activities.

C. HackerRank/Leetcode Link of the problem (If applicable):

D. Student's Github link of report and code upload, along with the readme file:

https://github.com/lakshmiprasanna200507/Neo-4j

E. Solution Steps/Algorithm:

1. Open Neo4j AuraDB and create a database instance.
2. Create nodes for Student, Faculty, Course, and Department with their respective properties.
3. Establish relationships such as `ENROLLED_IN`, `TEACHES`, `BELONGS_TO`, and `WORKS_IN`.
4. Insert sample data using Cypher queries.
5. Execute queries to retrieve student registrations, faculty assignments, and department-wise courses.
6. Verify the relationships and results using the Neo4j Browser visualization.
7. Save the Cypher queries and their outputs for documentation.
8. Continue BFS until the final cell is reached.
9. Return the minimum number of dice rolls.

12.f the final cell cannot be reached, return -1.

F. Code developed: from collections import deque

```
// 1. Create Student, Faculty, Course and Department nodes
CREATE
  (s1:Student {student_id: 1, name: 'Rahul Sharma', semester: 5}),
  (s2:Student {student_id: 2, name: 'Priya Reddy', semester: 5}),
  (s3:Student {student_id: 3, name: 'Arjun Kumar', semester: 5}),
  (s4:Student {student_id: 4, name: 'Sneha Rao', semester: 6}),
  (s5:Student {student_id: 5, name: 'Vikram Das', semester: 4}),

  (f1:Faculty {faculty_id: 101, name: 'Dr. Meera Sen'}),
  (f2:Faculty {faculty_id: 102, name: 'Dr. Ravi Kumar'}),
  (f3:Faculty {faculty_id: 103, name: 'Dr. Ananya Rao'}),

  (c1:Course {course_id: 201, course_name: 'Graph Analytics', credits: 4}),
  (c2:Course {course_id: 202, course_name: 'NoSQL Databases', credits: 3}),
  (c3:Course {course_id: 203, course_name: 'Database Systems', credits: 3}),
  (c4:Course {course_id: 204, course_name: 'Machine Learning', credits: 4}),

  (d1:Department {department_id: 301, name: 'Computer Science'}),
  (d2:Department {department_id: 302, name: 'Information Technology'});
// 2. Create relationships with appropriate directions
MATCH (s1:Student {student_id: 1}),
      (s2:Student {student_id: 2}),
```

```
(s3:Student {student_id: 3}),
    (s4:Student {student_id: 4}),
    (s5:Student {student_id: 5}),
    (f1:Faculty {faculty_id: 101}),
    (f2:Faculty {faculty_id: 102}),
    (f3:Faculty {faculty_id: 103}),
    (c1:Course {course_id: 201}),
    (c2:Course {course_id: 202}),
    (c3:Course {course_id: 203}),
    (c4:Course {course_id: 204}),
    (d1:Department {department_id: 301}),
    (d2:Department {department_id: 302})
CREATE
  (s1)-[:ENROLLED_IN {semester: 5, grade: 'A'}]->(c1),
  (s1)-[:ENROLLED_IN {semester: 5, grade: 'A'}]->(c2),
  (s2)-[:ENROLLED_IN {semester: 5, grade: 'B+'}]->(c1),
  (s2)-[:ENROLLED_IN {semester: 5, grade: 'A'}]->(c3),
  (s3)-[:ENROLLED_IN {semester: 5, grade: 'B+'}]->(c1),
  (s3)-[:ENROLLED_IN {semester: 5, grade: 'B'}]->(c2),
  (s4)-[:ENROLLED_IN {semester: 6, grade: 'A'}]->(c3),
  (s5)-[:ENROLLED_IN {semester: 4, grade: 'B'}]->(c4),
  (f1)-[:TEACHES {academic_year: '2026-2027'}]->(c1),
  (f1)-[:TEACHES {academic_year: '2026-2027'}]->(c2),
  (f2)-[:TEACHES {academic_year: '2026-2027'}]->(c3),
  (f3)-[:TEACHES {academic_year: '2026-2027'}]->(c4),
```

```
  (s1)-[:BELONGS_TO]->(d1),
  (s2)-[:BELONGS_TO]->(d1),
  (s3)-[:BELONGS_TO]->(d2),
  (s4)-[:BELONGS_TO]->(d1),
  (s5)-[:BELONGS_TO]->(d2),
  (f1)-[:WORKS_IN]->(d1),
  (f2)-[:WORKS_IN]->(d1),
  (f3)-[:WORKS_IN]->(d2),
  (c1)-[:BELONGS_TO]->(d1),
  (c2)-[:BELONGS_TO]->(d2),
  (c3)-[:BELONGS_TO]->(d2),
  (c4)-[:BELONGS_TO]->(d1);
// 3. Relationship properties are assigned above.
// 4. Display the complete graph
MATCH (n)-[r]->(m)
RETURN n, r, m;
// 5. Count nodes belonging to each label
MATCH (n)
RETURN labels(n) AS Label, count(n) AS TotalNodes
ORDER BY Label;

// 6. Display all relationship types and directions
MATCH (a)-[r]->(b)
RETURN DISTINCT
  labels(a) AS SourceLabel,
  type(r) AS RelationshipType,
```

```
labels(b) AS TargetLabel;

// 7. Retrieve all students studying in Semester 5

MATCH (s:Student)

WHERE s.semester = 5

RETURN s.name AS StudentName, s.semester AS Semester;

// 8. Find all courses with more than 3 credits

MATCH (c:Course)

WHERE c.credits > 3

RETURN c.course_name AS CourseName, c.credits AS Credits;

// 9. Retrieve courses taught by Dr. Meera Sen

MATCH (f:Faculty {name: 'Dr. Meera Sen'})-[:TEACHES]->(c:Course)

RETURN c.course_name AS CourseName;

// 10. Find all students enrolled in Graph Analytics

MATCH (s:Student)-[:ENROLLED_IN]->(c:Course {course_name: 'Graph
Analytics'})

RETURN s.name AS StudentName;

// 11. Find students belonging to Computer Science

MATCH (s:Student)-[:BELONGS_TO]->(d:Department {name: 'Computer
Science'})

RETURN s.name AS StudentName;

// 12. Find courses offered by Information Technology

MATCH (c:Course)-[:BELONGS_TO]->(d:Department {name: 'Information
Technology'})

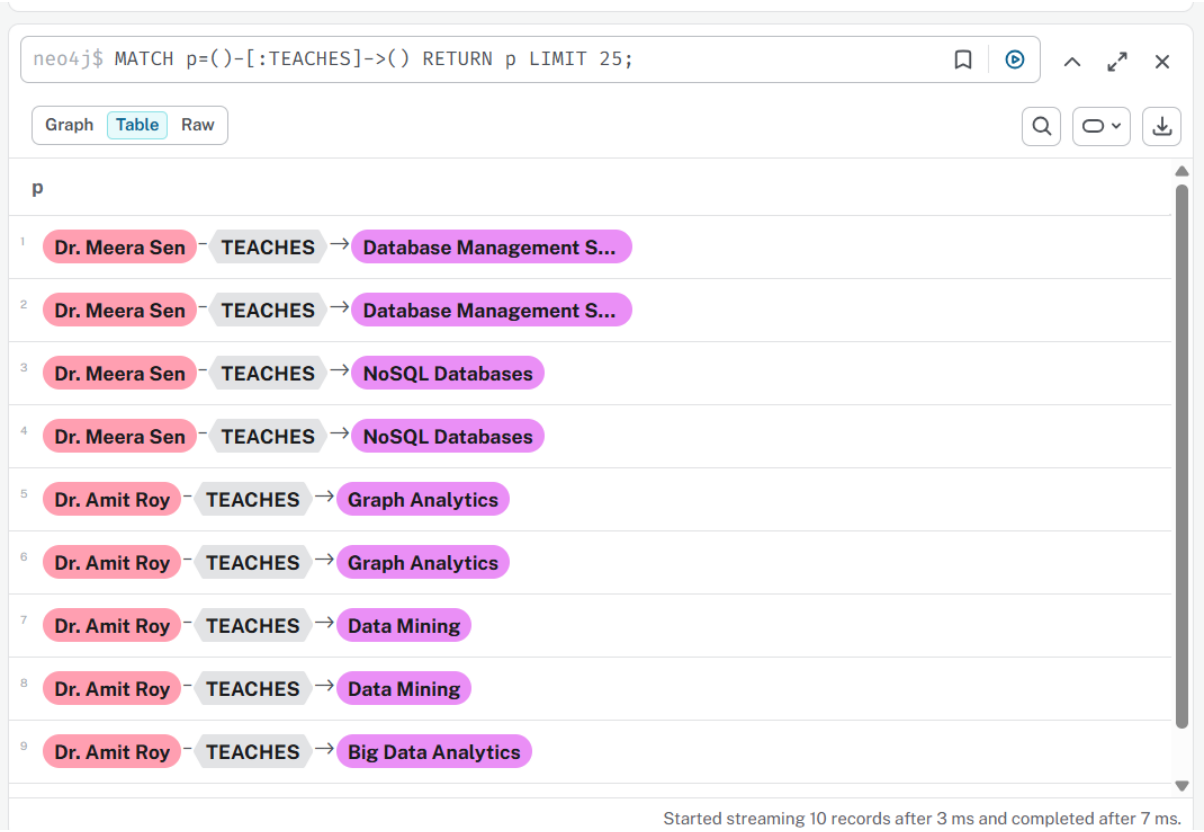
RETURN c.course_name AS CourseName;

// 13. Students enrolled in NoSQL Databases with semester and grade

MATCH (s:Student)-[r:ENROLLED_IN]->(c:Course {course_name: 'NoSQL
Databases'})
```

```
RETURN s.name AS StudentName,
       r.semester AS Semester,
       r.grade AS Grade;
// 14. Students enrolled in Graph Analytics and at least one other course
MATCH (s:Student)-[:ENROLLED_IN]->(c1:Course {course_name: 'Graph Analytics'}),
      (s)-[:ENROLLED_IN]->(c2:Course)
WHERE c1 <> c2
RETURN s.name AS StudentName,
       collect(DISTINCT c1.course_name) +
       collect(DISTINCT c2.course_name) AS Courses;
```

G. Output screenshot (Mandatory):



```
neo4j$ MATCH p=()-[:OFFERS]->() RETURN p LIMIT 25;
```

Graph | Table | Raw

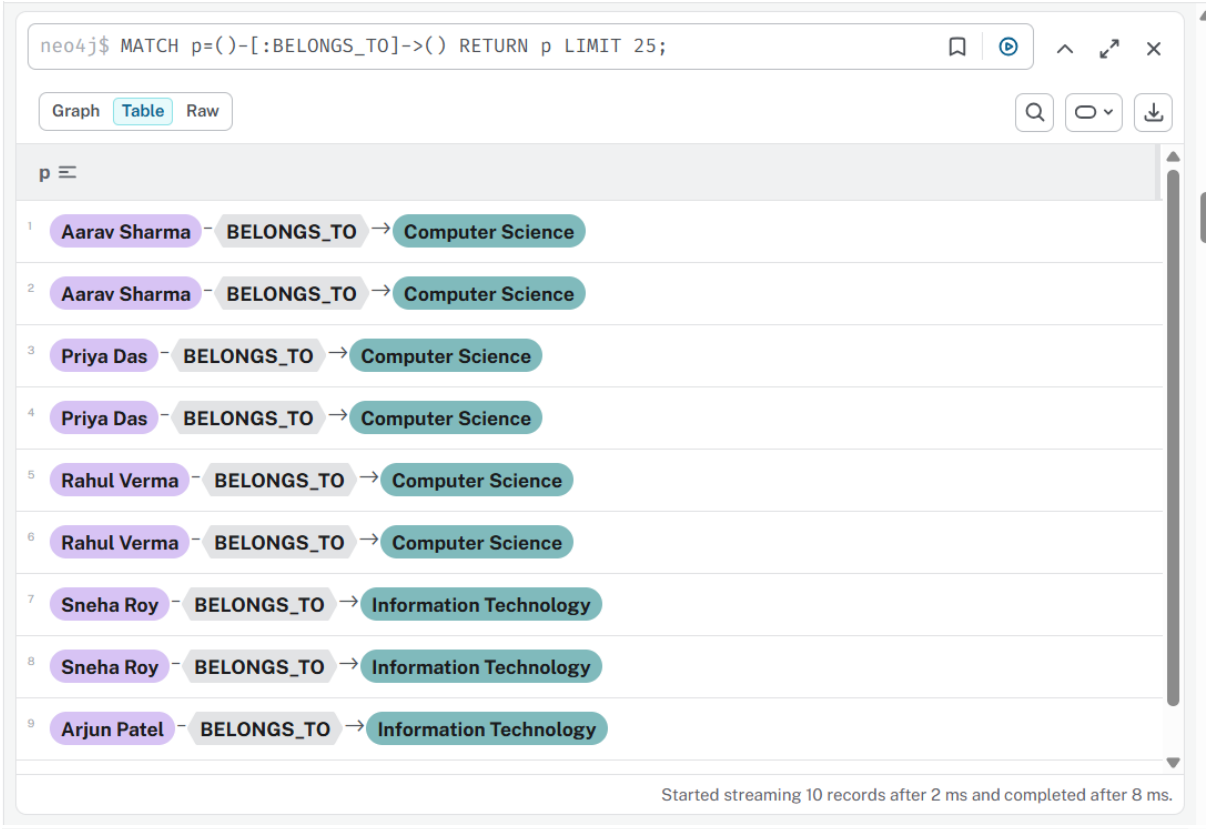
	p
1	Computer Science - OFFERS → Database Management S...
2	Computer Science - OFFERS → Database Management S...
3	Computer Science - OFFERS → Data Mining
4	Computer Science - OFFERS → Data Mining
5	Computer Science - OFFERS → NoSQL Databases
6	Computer Science - OFFERS → NoSQL Databases
7	Information Technology - OFFERS → Graph Analytics
8	Information Technology - OFFERS → Graph Analytics
9	Information Technology - OFFERS → Big Data Analytics

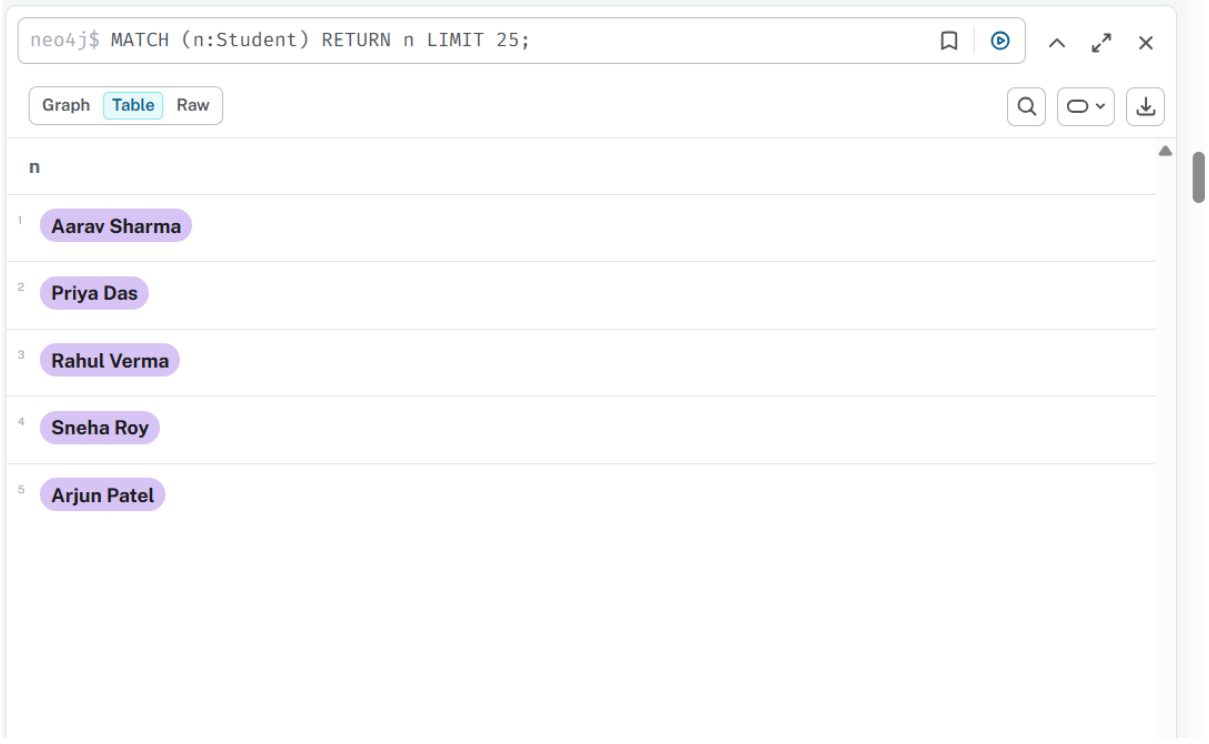
```
neo4j$ MATCH p=()-[:ENROLLED_IN]->() RETURN p LIMIT 25;
```

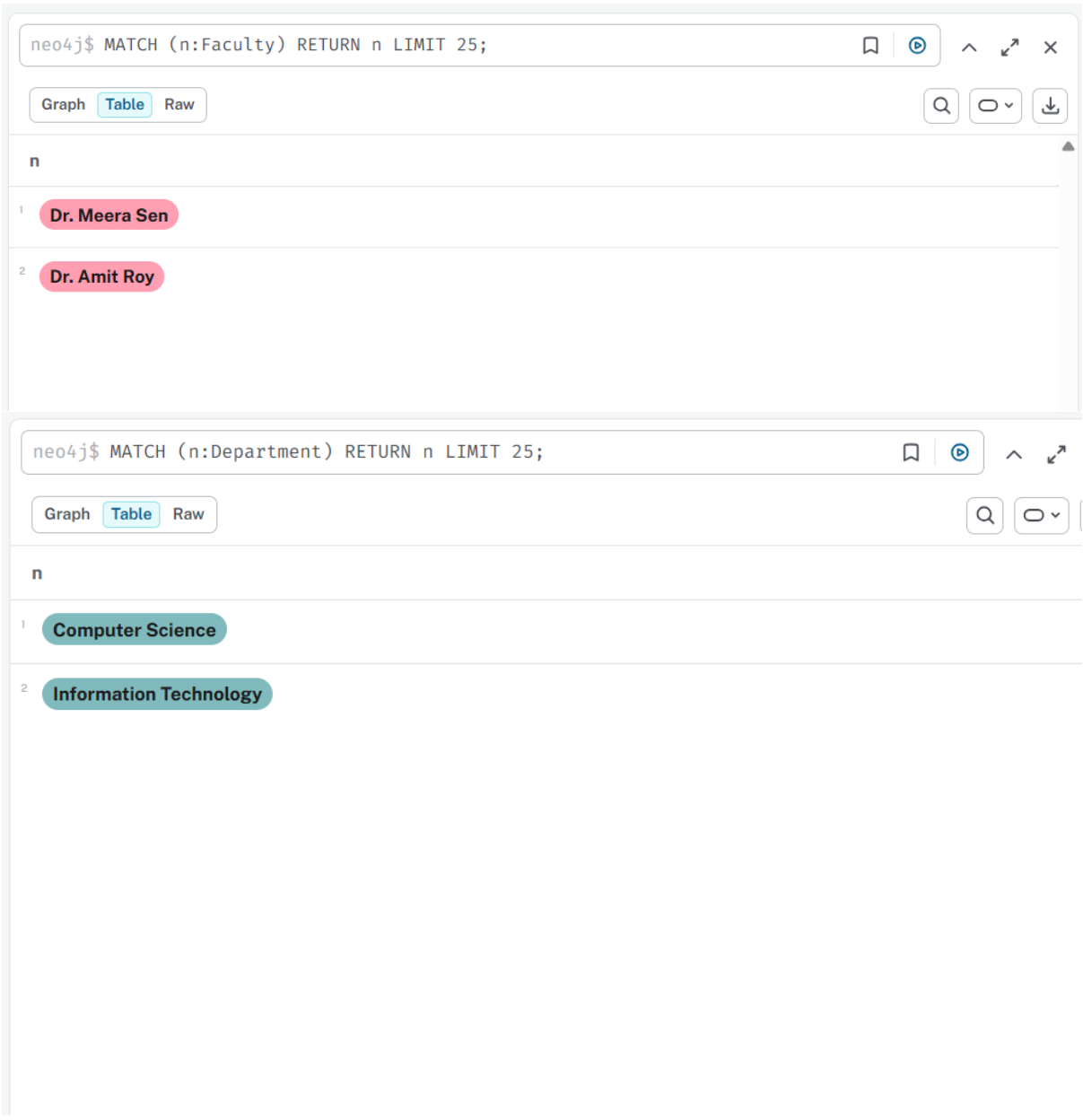
Graph | Table | Raw

	p
1	Aarav Sharma - ENROLLED_IN → Graph Analytics
2	Aarav Sharma - ENROLLED_IN → Graph Analytics
3	Aarav Sharma - ENROLLED_IN → Database Management S...
4	Aarav Sharma - ENROLLED_IN → Database Management S...
5	Aarav Sharma - ENROLLED_IN → NoSQL Databases
6	Aarav Sharma - ENROLLED_IN → NoSQL Databases
7	Priya Das - ENROLLED_IN → Graph Analytics
8	Priya Das - ENROLLED_IN → Graph Analytics
9	Priya Das - ENROLLED_IN → Data Mining

Started streaming 22 records after 2 ms and completed after 10 ms.


neo4j$ MATCH p=()-[:BELONGS_TO]->() RETURN p LIMIT 25;
Graph
Table
Raw
p
1 Aarav Sharma - BELONGS_TO → Computer Science
2 Aarav Sharma - BELONGS_TO → Computer Science
3 Priya Das - BELONGS_TO → Computer Science
4 Priya Das - BELONGS_TO → Computer Science
5 Rahul Verma - BELONGS_TO → Computer Science
6 Rahul Verma - BELONGS_TO → Computer Science
7 Sneha Roy - BELONGS_TO → Information Technology
8 Sneha Roy - BELONGS_TO → Information Technology
9 Arjun Patel - BELONGS_TO → Information Technology
Started streaming 10 records after 2 ms and completed after 8 ms.


neo4j$ MATCH (n:Student) RETURN n LIMIT 25;
Graph
Table
Raw
n
1 Aarav Sharma
2 Priya Das
3 Rahul Verma
4 Sneha Roy
5 Arjun Patel


neo4j$ MATCH (n:Faculty) RETURN n LIMIT 25;
Graph
Table
Raw
n
1
Dr. Meera Sen
2
Dr. Amit Roy
neo4j$ MATCH (n:Department) RETURN n LIMIT 25;
Graph
Table
Raw
n
1
Computer Science
2
Information Technology

```
neo4j$ MATCH (n:Course) RETURN n LIMIT 25;
```

Graph | Table | Raw

	n
1	Graph Analytics
2	Database Management S...
3	Data Mining
4	Big Data Analytics
5	NoSQL Databases

```
neo4j$
```

	Student	OtherCourses
1	"Aarav Sharma"	["Database Management Systems", "Database Management Systems", "NoSQL Databases", "NoSQL Databases", "Database Management Systems", "Database Management Systems", "NoSQL Databases", "NoSQL Databases"]
2	"Priya Das"	["Data Mining", "Data Mining", "Data Mining", "Data Mining"]
3	"Arjun Patel"	["Big Data Analytics", "Big Data Analytics", "Big Data Analytics", "Big Data Analytics"]

Started streaming 3 records after 225 ms and completed after 230 ms.

```
neo4j$ MATCH (s:Student)-[r:ENROLLED_IN]->(c:Course {title:'NoSQL Databases'}
```

Table | Raw

	Student	Semester	Grade
1	"Aarav Sharma"	5	"Not Published"
2	"Aarav Sharma"	5	"Not Published"
3	"Sneha Roy"	3	"Not Published"
4	"Sneha Roy"	3	"Not Published"

```
neo4j$ MATCH (f:Faculty {name:'Dr. Meera Sen'})-[:TEACHES]->(c:Course) RETURN c.t
```

```
MATCH (f:Faculty {name:'Dr. Meera Sen'})-[:TEACHES]→(c:Course)
```

```
MATCH (s:Student)-[:ENROLLED_IN]→(c:Course {title:'Graph Analytics'})
```

```
MATCH (s:Student)-[:BELONGS_TO]→(d:Department {name:'Computer Science'})
```

```
MATCH (d:Department {name:'Information Technology'})-[:OFFERS]→(c:Course)
```

```
neo4j$ MATCH (c:Course) WHERE c.credits > 3 RETURN c.title AS Course, c.credi
```

Table Raw

	Course	Credits
1	"Database Management Systems"	4
2	"Big Data Analytics"	4

Started streaming 2 records in less than 1ms and completed after 5 ms.


```
neo4j$ MATCH (c:Course) WHERE c.credits > 3 RETURN c.title AS Course, c.credi
```

Table Raw

	Course	Credits
1	"Database Management Systems"	4
2	"Big Data Analytics"	4

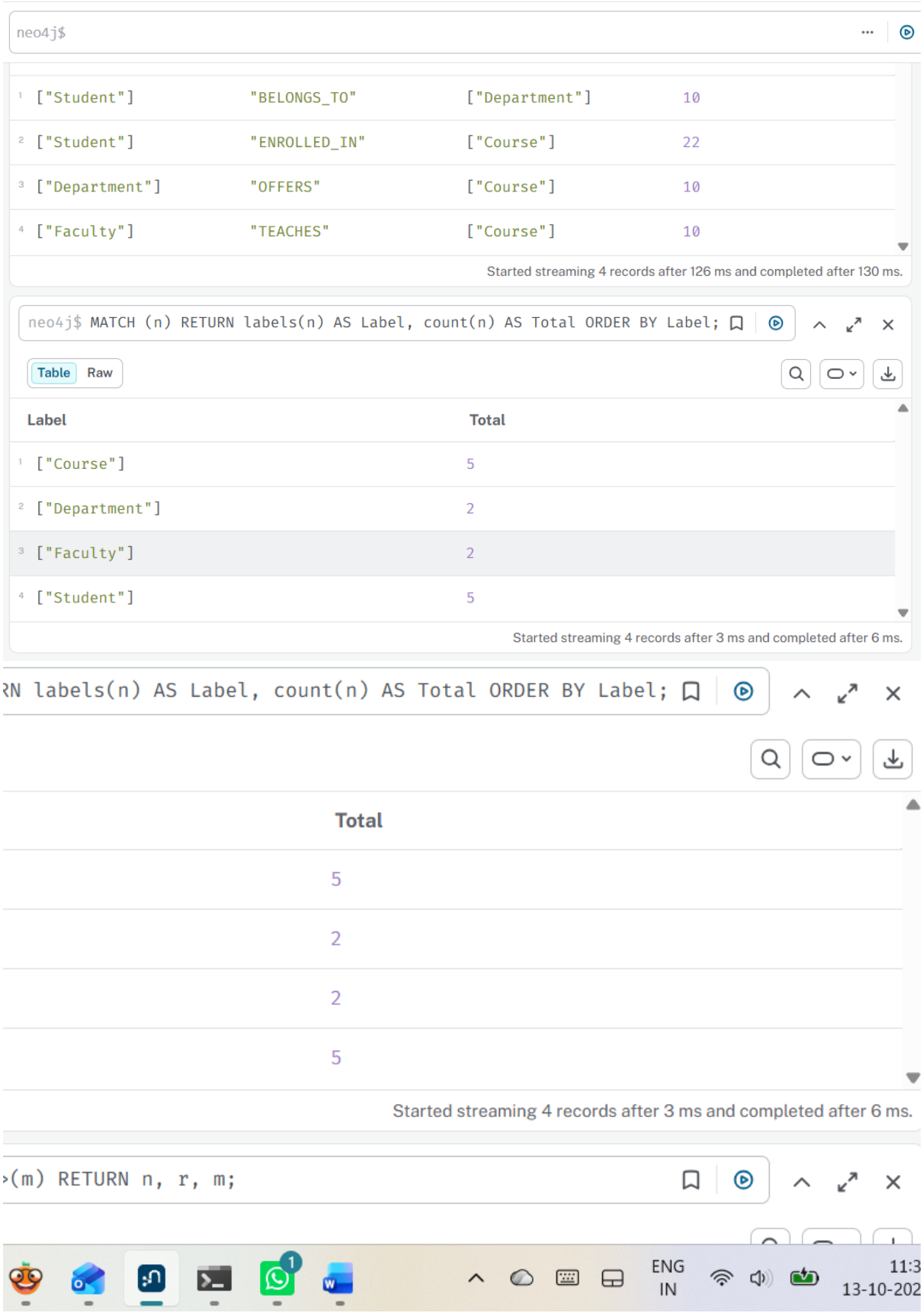
Started streaming 2 records after 151 ms and completed after 155 ms.

```
neo4j$ MATCH (s:Student) WHERE s.semester = 5 RETURN s.name AS Student, s.sem
```

Table Raw

	Student	Semester
1	"Aarav Sharma"	5
2	"Priya Das"	5
3	"Arjun Patel"	5

Started streaming 3 records after 78 ms and completed after 80 ms.



```
neo4j$
```

1	["Student"]	"BELONGS_TO"	["Department"]	10
2	["Student"]	"ENROLLED_IN"	["Course"]	22
3	["Department"]	"OFFERS"	["Course"]	10
4	["Faculty"]	"TEACHES"	["Course"]	10

Started streaming 4 records after 126 ms and completed after 130 ms.

```
neo4j$ MATCH (n) RETURN labels(n) AS Label, count(n) AS Total ORDER BY Label;
```

Table Raw

	Label	Total
1	["Course"]	5
2	["Department"]	2
3	["Faculty"]	2
4	["Student"]	5

Started streaming 4 records after 3 ms and completed after 6 ms.

```
RN labels(n) AS Label, count(n) AS Total ORDER BY Label;
```

Total
5
2
2
5

Started streaming 4 records after 3 ms and completed after 6 ms.

```
>(m) RETURN n, r, m;
```

ENG IN 11:3 13-10-202

```
neo4j$ MATCH (n)-[r]->(m) RETURN n, r, m;
```

Graph | Table | Raw

	n	r	m
1	Aarav Sharma	ENROLLED_IN	Graph Analytics
2	Aarav Sharma	ENROLLED_IN	Graph Analytics
3	Aarav Sharma	ENROLLED_IN	Database Management S...
4	Aarav Sharma	ENROLLED_IN	Database Management S...
5	Aarav Sharma	ENROLLED_IN	NoSQL Databases
6	Aarav Sharma	ENROLLED_IN	NoSQL Databases
7	Aarav Sharma	BELONGS_TO	Computer Science
8	Aarav Sharma	BELONGS_TO	Computer Science

Started streaming 52 records in less than 1ms and completed after 14 ms.

```
neo4j$ MATCH ()-[r]->() RETURN type(r) AS RelationshipType, count(r) AS Total
```

Table | Raw

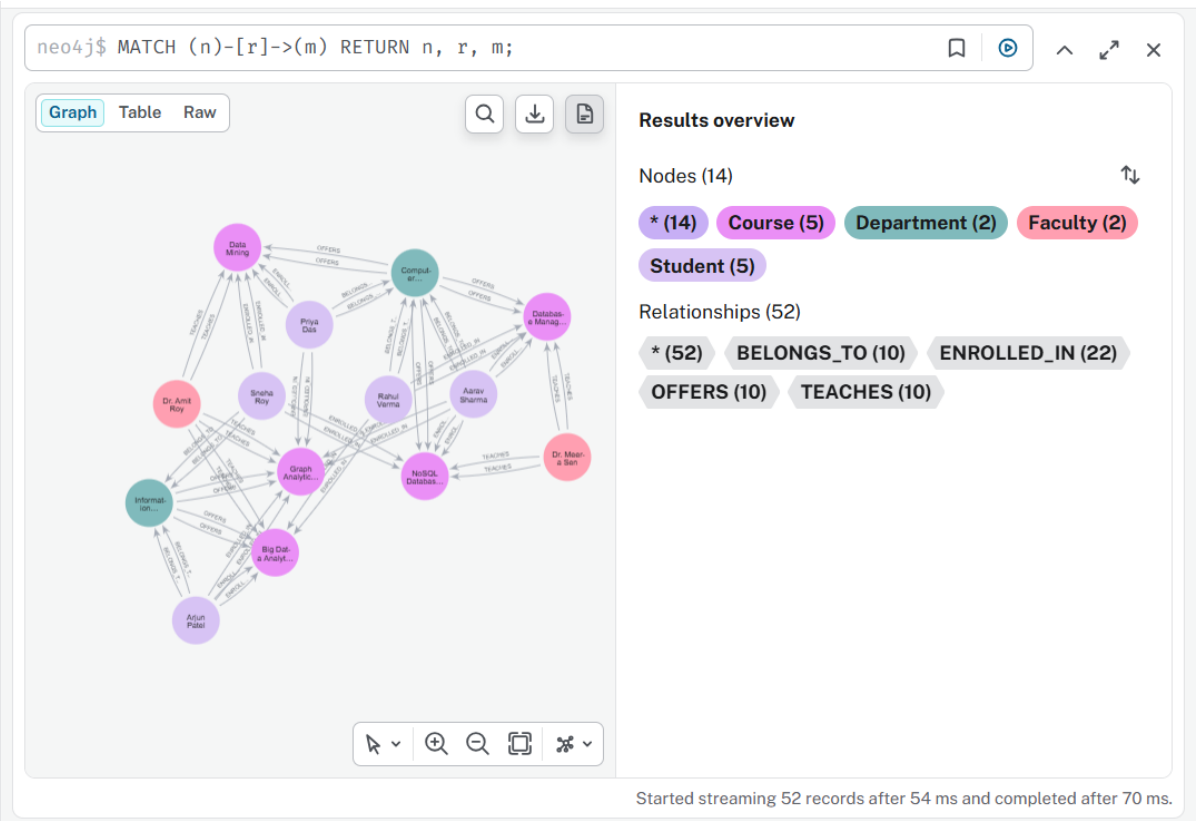
	RelationshipType	Total
1	"BELONGS_TO"	10
2	"ENROLLED_IN"	22
3	"OFFERS"	10
4	"TEACHES"	10

Started streaming 4 records after 200 ms and completed after 206 ms.

Table Raw

Label	Total
["Course"]	5
["Department"]	2
["Faculty"]	2
["Student"]	5

Started streaming 4 records after 10 ms and completed after 12 ms.



H. Observation:

The University Course Registration System was successfully implemented using Neo4j and the Cypher query language. Student, Faculty, Course, and Department nodes were created, and relationships such as ENROLLED_IN, TEACHES, BELONGS_TO, and WORKS_IN were established. The graph database successfully represents the connections between university entities and allows efficient retrieval of course registration and faculty information.